INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LABORATORY INVESTIGATION & FORENSIC EVIDENCE REPORT

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

Lab Title: Digital Forensics Lab: Steganography & Hidden Data Detection	**Course Name:** Digital Forensics (Module 08)
Student Name: Almustapha Yusuf	**Instructor Name:** Aminu Idris
Student ID: fwsd2511343@students.icdfa.edu.ng	**Date of Submission:** 05/09/2026
Environment: Kali Linux (Rolling Release)	**Report Version:** Version 1.0 (Final Release)

2.2 Executive Summary

This lab demonstrated how steganography can be used to conceal data within an image and how forensic analysts can detect and recover it. Using the Steghide tool, a text file was successfully embedded into a BMP carrier image using the password 1234. The extraction process verified data integrity using SHA-256 hashes and diff. The lab also compared the original and stego images, revealing identical file sizes and initial bytes (xxd) but completely different cryptographic hashes, proving the use of Least Significant Bit (LSB) substitution. Finally, StegSeek was used with a controlled wordlist to successfully recover the password 1234, demonstrating a standard password-recovery workflow.

2.3 Lab Objectives

- Explain the difference between steganography and encryption.
- Describe insertion and substitution-based hiding techniques, including LSB.
- Prepare a carrier image and secret evidence file.
- Use Steghide to embed and extract concealed data.
- Calculate and compare cryptographic hashes.
- Perform a controlled password dictionary test using StegSeek.
- Document all activities in a repeatable forensic report.

2.4 Tools and Resources Used

- **Steghide:** Used to embed and extract hidden data within BMP images using LSB algorithms.
- **StegSeek (v0.6):** Lightning-fast cracker used for steganographic password recovery and dictionary attacks.
- **xxd:** Hex viewer utility used to inspect and compare file signatures, bitmap headers, and initial bytes.
- **sha256sum:** Cryptographic utility used to calculate and verify data integrity and alteration proof.
- **Kali Linux:** Primary digital forensics operating platform hosting all forensic execution suites.

2.5 Methodology

- **Part A (Workspace & Carrier Baseline):** Created dedicated working directory structure (ICDFA-Steganography-Lab/{evidence,results,screenshots,scripts,reports}), confirmed carrier image attributes (tower_original_image_for_lab.bmp, 8.9M, 1365x2265x24 BMP), and computed SHA-256 baseline.

- **Part B (Secret Evidence Creation):** Created secret.txt containing student identification and controlled evidentiary message, followed by immediate SHA-256 hash calculation.
- **Part C (Steganographic Embedding):** Embedded secret.txt into the carrier bitmap using steghide embed -p 1234, producing tower_stego_lab4.bmp, and recorded output file size and cryptographic hash.
- **Part D (Forensic Extraction & Integrity Verification):** Extracted hidden payload using steghide extract -p 1234 into extracted/extracted_secret.txt. Verified data integrity against source evidence using diff and sha256sum.
- **Part E (Comparative Artifact Analysis):** Executed structured comparisons across original carrier vs stego image: file sizes (ls -lh), SHA-256 hashes, initial hex bytes (xxd head -3), and human-readable ASCII/Unicode strings (strings head -20).
- **Part F (Controlled Dictionary Recovery):** Constructed a target wordlist (lab4_wordlist.txt) containing passphrase 1234, and ran StegSeek against tower_stego_lab4.bmp to achieve automated passphrase discovery and extraction.
- **Part G (Independent Task & Validation):** Authored custom hidden note (Almustapha_Yusuf_hidden_note.txt), computed pre-embed hash, embedded into carrier image as independent_stego.bmp with passphrase 1234, extracted, validated bitwise match, and completed StegSeek dictionary attack recovery.

2.6 Screenshots and Evidence

The following documented figures provide cryptographic and visual chain-of-custody evidence verifying each methodology phase performed inside the Kali Linux environment.

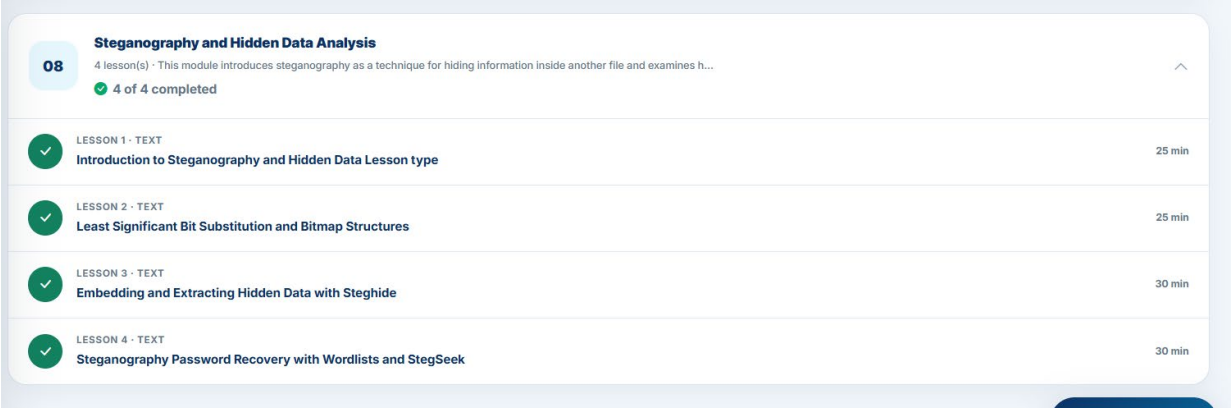

Figure 1: *Module Completion Status* — ICDFA LMS showing 4 of 4 completed lessons in Module 08.

Figure 2: *Directory Setup & Carrier Baseline Hashing (Part A)* — Verification of tower_original_image_for_lab.bmp attributes and SHA-256 calculation.

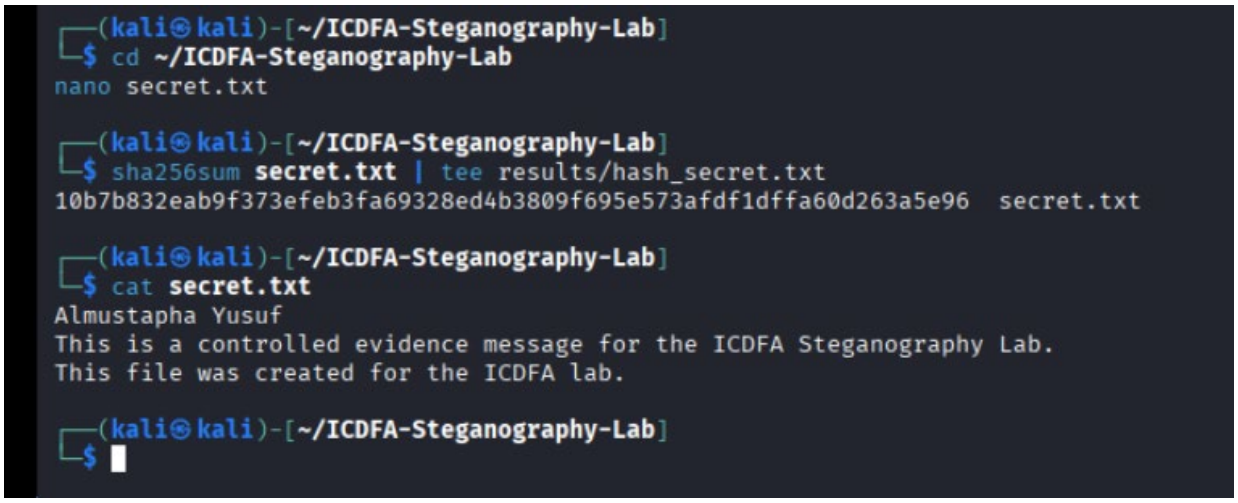

Figure 3: *Secret Evidence Creation & Hash Generation (Part B)* — Authoring of secret.txt with user identification and recording SHA-256 checksum.

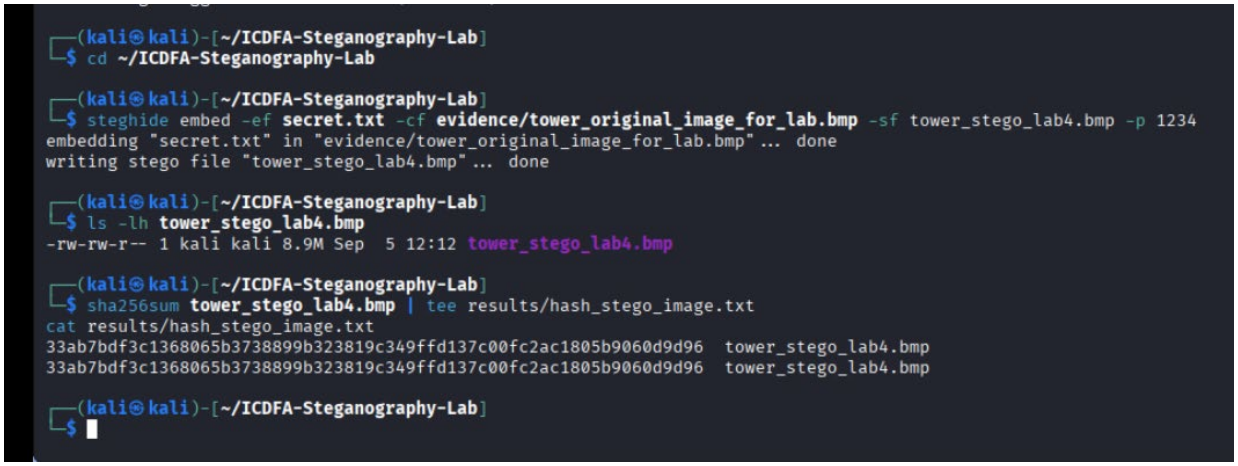

Figure 4: *Steganographic Embedding via Steghide (Part C)* — Embedding secret.txt with passphrase '1234' producing tower_stego_lab4.bmp.

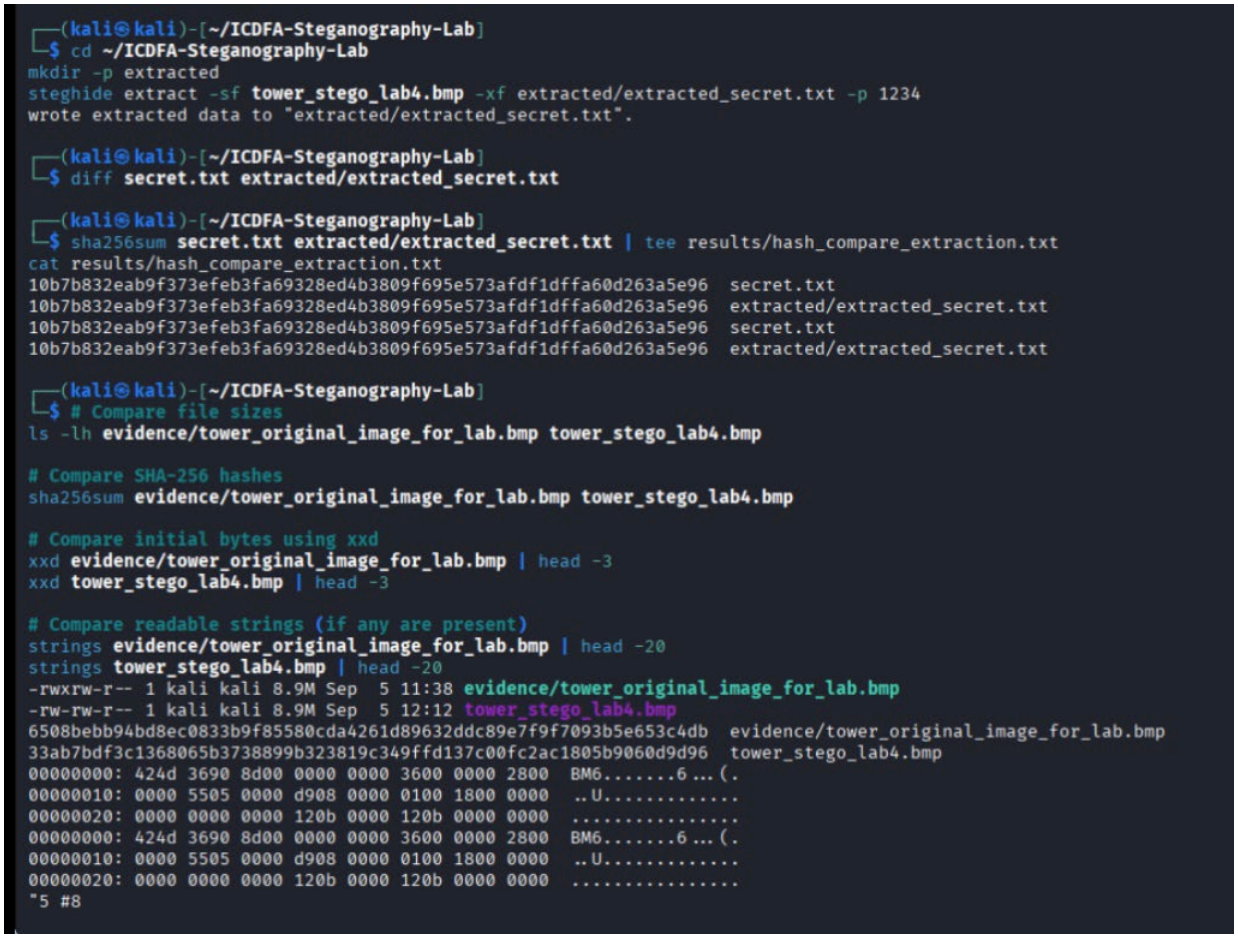

Figure 5: *Payload Extraction & Comparative Verification (Parts D & E)* — Side-by-side diff, SHA-256 match, and xxd comparison showing identical BM headers.

Figure 6: *String Extraction Analysis (Part E)* — Inspection of high-entropy embedded payload bytes via strings output.

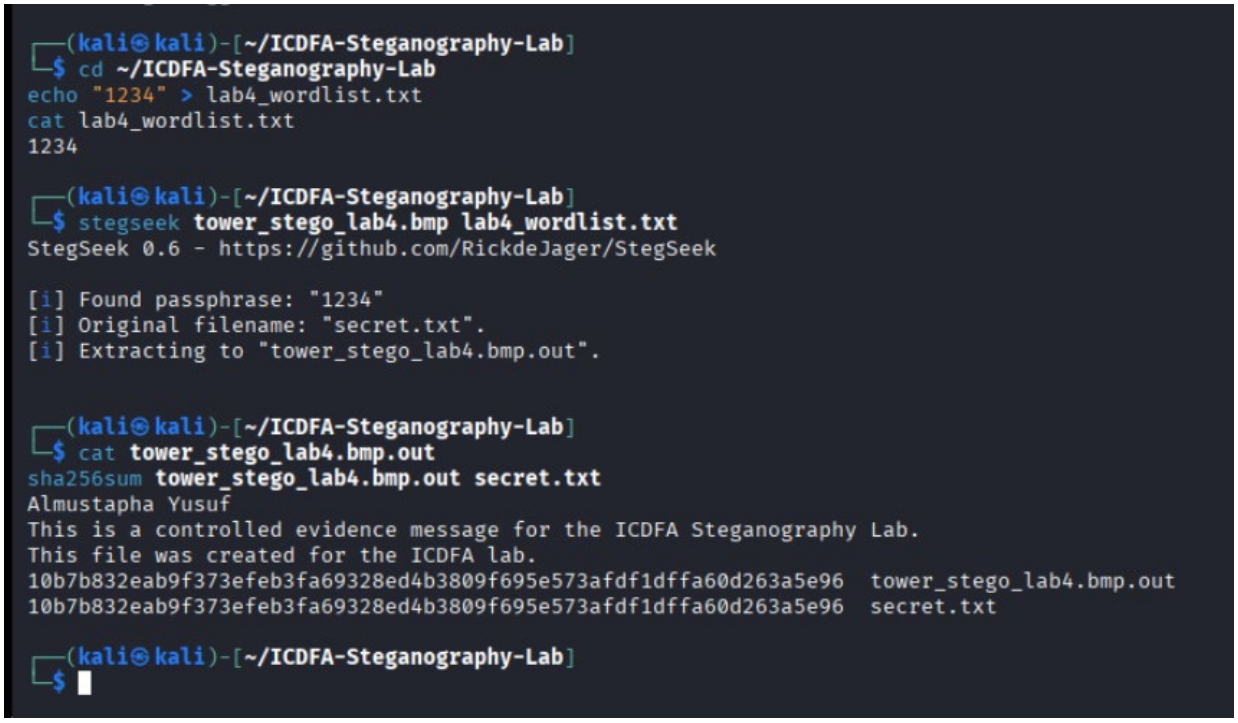

Figure 7: *StegSeek Password Dictionary Cracking (Part F)* — Rapid discovery of passphrase '1234' and integrity verification of tower_stego_lab4.bmp.out.

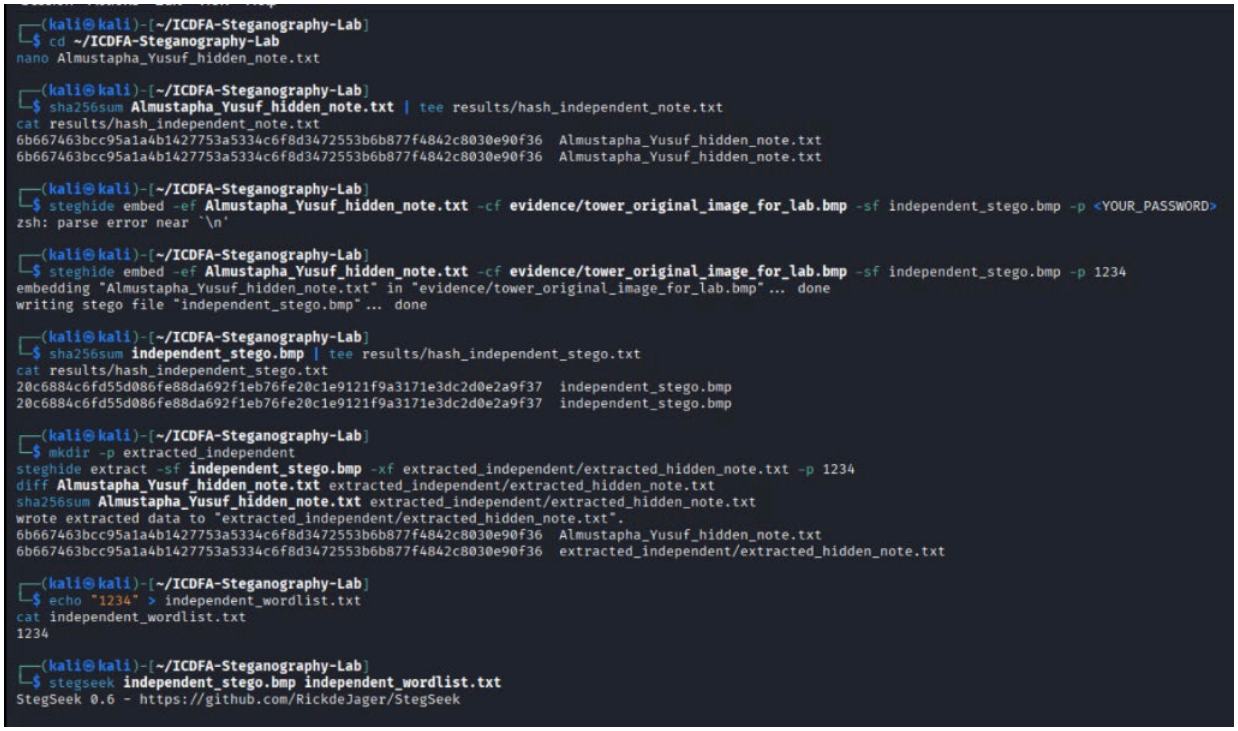

Figure 8: *Independent Task Execution & Syntax Resolution (Part G)* — Creation of Almustapha_Yusuf_hidden_note.txt, resolving zsh bracket parse error, and embedding.

Figure 9: *Independent Evidence StegSeek Recovery & Hash Validation (Part G)* — StegSeek extraction of independent note and identical SHA-256 hash match confirmation.

2.7 Analysis and Findings

A forensic comparison between the unaltered carrier file, the initial stego image, and the independent stego image demonstrates key properties of Least Significant Bit (LSB) steganography:

Artifact File	Size	Cryptographic SHA-256 Hash	Header Signature (xxd)
tower_original_image_for_lab.bmp (Original Carrier)	8.9 MB	`6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db`	`424d 3690 8d00... (BM6.)`
tower_stego_lab4.bmp (Steghide Embedded)	8.9 MB	`33ab7bdf3c1368065b3738899b323819c349ffd137c00fc2ac1805b9060d9d96`	`424d 3690 8d00... (BM6.)`
independent_stego.bmp (Independent Task)	8.9 MB	`20c6884c6fd55d086fe88da692f1eb76fe20c1e9121f9a3171e3dc2d0e2a9f37`	`424d 3690 8d00... (BM6.)`

- **Structural Header Consistency:** The xxd utility verified that the first three 16-byte lines (0x00000000 to 0x00000020) containing the 54-byte BMP header, DIB header, width, height, and color bit depth (24-bit) were completely unaltered across carrier and stego files.
- **Cryptographic Divergence Proving LSB Substitution:** While byte counts remained perfectly static at 8.9 MB, the cryptographic hashes changed completely (6508... to 33ab... and 20c6...). This proves that data was substituted inside existing pixel least-significant bits rather than appended via an end-of-file (EOF) injection.
- **Payload Integrity:** The diff comparison and identical SHA-256 digests (10b7... and 6b66...) between original input files and extracted files confirmed 100% data integrity without byte degradation or corruption.
- **StegSeek Performance:** StegSeek parsed the stego bitmaps and extracted the hidden payload within milliseconds using dictionary matching, highlighting the extreme risk of utilizing simple, predictable passphrases.

2.8 Challenges and Solutions

- **Challenge:** During the independent steganography task, executing the command steghide embed -ef Almustapha_Yusuf_hidden_note.txt -cf evidence/tower_original_image_for_lab.bmp -sf independent_stego.bmp -p <YOUR_PASSWORD> resulted in an immediate shell parsing error: 'zsh: parse error

near \n'. The zsh interpreter evaluated the angular brackets '<' and '>' as file input/output redirection operators rather than a string literal argument.

- **Solution:** Identified that <YOUR_PASSWORD> was intended as a template placeholder. Substituted the placeholder with the intended literal password ('-p 1234'). Upon re-execution, the Steghide command completed successfully without errors.

2.9 Conclusion

This lab successfully demonstrated the core principles of steganography, LSB substitution, and forensic password recovery. It highlighted that while steganographic carrier images remain visually indistinguishable from original media and retain identical file lengths and headers, their cryptographic hash values diverge entirely. This divergence offers forensic investigators incontrovertible proof of alteration. Furthermore, the lab proved that specialized cracking engines such as StegSeek render simple passphrases ineffective, making stego-analysis a vital component in modern digital forensics workflows.

2.10 Recommendations

- **Enforce High-Entropy Passphrases:** Always employ complex, high-entropy passphrases (16+ alphanumeric/symbolic characters) when embedding data to resist high-velocity dictionary recovery tools like StegSeek.
- **Establish Dual-Hash Forensic Baselines:** Forensic examiners must compute SHA-256 and SHA-512 hashes before and after any extraction workflow to provide tamper-proof evidentiary integrity in legal proceedings.
- **Develop Suspect-Specific Wordlists:** Digital forensic analysts encountering steganographic artifacts should generate targeted, case-specific dictionaries (combining OSINT, usernames, case dates, and leetspeak mutations) to maximize cracking efficiency.

2.11 References

- Hetzl, S. (2003). Steghide: A steganography program. SourceForge. Retrieved from https://steghide.sourceforge.net/
- De Jager, R. (2020). StegSeek: Fast Steghide cracker using precomputed hashes. GitHub. Retrieved from https://github.com/RickdeJager/StegSeek
- National Institute of Standards and Technology (NIST). (2021). Guide to Digital Forensics (Special Publication 800-86). U.S. Department of Commerce.
- International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab Assignment Template Guide & Module 08 Courseware.